BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 304 (July 2025)

Digital Electronics Laboratory

Final Project Report

Section: C2 Group: 05

Accelerometer-Driven Display Rotator and Tilt Level Meter

Course Instructors:

Tanushri Medha Kundu , Lecturer
Md. Mostaqul Islam, Part-Time Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. <u>Type the student ID and name, and put your signature</u>. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: মুশফিক **Full Name: Mushfiqur Rahman** **Student ID: 2106181**	**Signature:** Chayan. **Full Name: Chayan Paul** **Student ID: 2106182**
Signature: Shamin **Full Name: Shamin Yeaser** **Student ID: 2106183**	**Signature:** Kayes **Full Name: Kayes Sami Malitha** **Student ID: 2106184**

Table of Contents

1 Abstract 1
2 Introduction 1
3 Design 2
3.1 Problem Formulation (PO(b)) 2
3.1.1 Identification of Scope 2
3.1.2 Literature Review 3
3.1.3 Formulation of Problem 3
3.1.4 Analysis 5
3.2 Design Method (PO(a)) 6
3.3 CAD/Hardware Design 15
3.4 Full Source Code of Firmware 16
4 Implementation 16
4.1 Description 16
5 Design Analysis and Evaluation 18
5.1 Novelty 18
5.2 Design Considerations (PO(c)) 18
5.3 Investigations (PO(d)) 19
5.3.1 Design of Experiment 19
5.3.2 Data Collection 19
5.3.3 Results and Analysis 19
5.3.4 Interpretation and Conclusions on Data 19
5.4 Limitations of Tools (PO(e)) 19
5.5 Impact Assessment (PO(f)) 19
5.5.1 Assessment of Societal and Cultural Issues 19
5.5.2 Assessment of Health and Safety Issues 20
5.5.3 Assessment of Legal Issues 20
5.6 Sustainability Evaluation (PO(g)) 20
5.7 Ethical Issues (PO(h)) 20
6 Reflection on Individual and Team work (PO(i)) 20
6.1 Individual Contribution of Each Member 21
6.2 Mode of TeamWork 21
6.3 Diversity Statement of Team 22

6.4 Log Book of Project Implementation 22
7 Communication to External Stakeholders (PO(j)) 22
7.1 Executive Summary 22
7.2 User Manual 23
7.3 Github Link 24
7.4 YouTube Link 24
8 Future Work (PO(l)) 24
9 References 24

1 Abstract

This project focuses on designing and implementing a system that uses an accelerometer with the Nexys A7-100T FPGA board to create a display rotator and tilt level meter. A 3-axis accelerometer is connected to the FPGA through the SPI protocol to continuously collect real-time acceleration data along the X, Y, and Z directions. This data is then processed using fixed-point arithmetic along with digital filtering methods to determine the orientation angles.

To improve accuracy, a moving average filter is used to minimize noise in the sensor readings. The roll angle is calculated using a hardware-efficient CORDIC algorithm and is shown on a 7-segment display as a rotating character. Meanwhile, the pitch angle controls an LED-based digital spirit level that indicates forward and backward tilt.

Overall, the system demonstrates real-time data processing, hardware-based mathematical operations, and digital communication, making it a good example of an embedded FPGA-based sensor application.

2 Introduction

Modern embedded systems often need to detect orientation in real time for applications like robotics, smartphones, gaming, and navigation. This project aims to design a system that measures tilt using an accelerometer and displays the orientation through LEDs and a 7-segment display.

Fig 2.1: Nexys A7 100T Board

This problem is considered a complex engineering problem because it combines multiple areas such as digital design, signal processing, and embedded systems. It also requires real-time operation, must work within hardware limitations like using fixed-point instead of floating-point arithmetic, and needs proper noise filtering and reliable sensor data collection.

There are several alternative approaches. A microcontroller-based system (like Arduino) is easier to implement but slower than an FPGA. Software solutions using tools like MATLAB or Python can be very accurate but are not suitable for real-time hardware applications. Using floating-point processors can improve accuracy, but they require more hardware resources.

An FPGA-based solution is selected because it offers parallel processing, faster operation, predictable timing, and an efficient hardware-based implementation.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

This project focuses on developing an FPGA-based system that can measure and display orientation using an accelerometer. The scope of the project covers several important tasks related to hardware interfacing, data processing, and output visualization.

Project Scope:

- **Interfacing a 3-axis accelerometer using SPI:**
  The system establishes communication between the FPGA and a 3-axis accelerometer through the SPI protocol. This allows the FPGA to continuously receive acceleration data along the X, Y, and Z axes in real time.
- **Processing real-time sensor data:**
  The incoming data from the accelerometer is processed instantly within the FPGA. This ensures that any change in orientation is detected and reflected without noticeable delay.
- **Implementing fixed-point mathematical operations:**
  Since FPGAs have limited resources for complex calculations, fixed-point arithmetic is used instead of floating-point. This helps in performing mathematical computations efficiently while maintaining acceptable accuracy.
- **Displaying orientation using LEDs and a 7-segment display:**
  The processed data is converted into a visual form. LEDs are used to indicate tilt direction (like a digital level), while a 7-segment display shows orientation information such as angle or rotation.

Limitations:

- **No floating-point operations:**
  The system avoids floating-point calculations due to high resource usage, which may slightly reduce precision in angle computation.
- **Limited FPGA resources:**
  The FPGA has constraints in terms of logic units and memory, which restricts the complexity of algorithms that can be implemented.

- **Approximate angle calculation:**
  Due to the use of fixed-point arithmetic and simplified algorithms, the calculated orientation angles are approximate rather than perfectly accurate.

3.1.2 Literature Review

Accelerometer-based tilt detection is widely used in embedded systems. Traditional methods compute angles using:

- Trigonometric functions:
  → Roll ≈ arctan(Y/Z)
  → Pitch ≈ arctan($X/\sqrt{Y^2 + Z^2}$) [1]

However, direct computation is expensive in hardware. Therefore:

- **CORDIC algorithm** is used for efficient angle computation
- **Moving average filters** are used for noise reduction

Previous implementations use:

- Microcontrollers with software libraries
- DSP processors for signal filtering

This project implements everything in pure hardware using Verilog, which improves speed and efficiency.

3.1.3 Formulation of Problem

In this project, the main goal is to determine the orientation of a system using data obtained from a 3-axis accelerometer and display the results in a user-friendly way.

Given

- The accelerometer provides acceleration values along three axes: X, Y, and Z
- Each of these values is represented as a 16-bit signed number, meaning they can represent both positive and negative accelerations depending on direction

Objective

The system is designed to perform the following tasks:

- **Compute roll and pitch angles:**
  Using the acceleration values, the system calculates the orientation of the device in terms of roll (side-to-side tilt) and pitch (forward-backward tilt)

- **Display roll angle on a 7-segment display:**
  The calculated roll angle is converted into a suitable format and shown visually using a 7-segment display, as a rotating or changing character (∩).

Fig 3.1: Special Character to be rotated in the display

- **Display pitch angle using LEDs:**
  The pitch angle is represented through LEDs, where different LED patterns indicate the direction and level of tilt (forward or backward)

Mathematical Formulation

The following figure shows the X, Y & Z axis orientation of the Nexys A7 100T FPGA board, which was found by analyzing the accelerometer data using the traditional roll pitch angle formula.

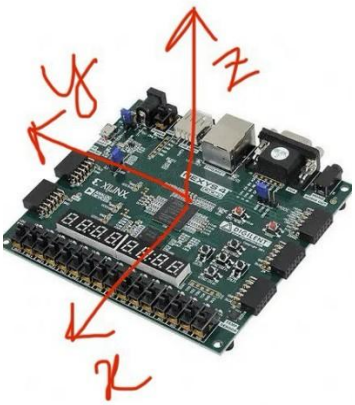

Fig 3.2: Nexys A7 100T Board Orientation

The orientation angles are derived using trigonometric relationships:

- **Roll angle:**

$$\theta roll = \tan{-1}(Y/X)$$

  This equation calculates the roll angle based on the ratio of Y-axis to Z-axis acceleration

- **Pitch angle:**

  $$\theta pitch = \tan^{-1}(X/Z)$$

  This equation determines the pitch angle using the ratio of X-axis to Z-axis acceleration

Assumptions:

- The +ve X direction is considered normally earthward.
- When display rotation is observed, the pitch level meter is ignored. This simplifies the equation for the pitch angle.

Constraints

The design must operate under the following limitations:

- **Use of fixed-point arithmetic:**
  All calculations must be performed using fixed-point representation instead of floating-point to reduce hardware complexity and resource usage.
- **Real-time processing requirement:**
  The system must continuously process incoming data and update outputs instantly without noticeable delay.
- **Limited hardware resources:**
  The FPGA has finite logic elements and memory, so the algorithms must be optimized to fit within these constraints while still maintaining acceptable performance and accuracy.

3.1.4 Analysis:

In this project, several challenges arise when processing accelerometer data and displaying orientation angles in real time. These challenges and their solutions are outlined below:

Challenges:

- **Noisy sensor data:**
  Accelerometers inherently produce small fluctuations and noise[2], which can make direct angle calculation unstable or inaccurate. Filtering is needed to smooth the data.
- **Hardware-intensive calculations:**
  Computing division and trigonometric functions, such as arctangent for roll and pitch angles, is computationally expensive on an FPGA if done directly. Efficient hardware methods are required.
- **Continuous real-time data stream:**
  The accelerometer continuously outputs data, and the system must process this stream without missing any samples. Proper synchronization and timing are necessary to ensure reliable processing.

Solutions:

- **Moving average filter:**
  Applying a moving average filter reduces noise in the sensor readings, resulting in smoother and more accurate angle calculations. [3]
- **CORDIC algorithm:**
  The CORDIC (Coordinate Rotation Digital Computer) method efficiently computes trigonometric functions like arctangent using iterative shift-and-add operations, making it suitable for FPGA implementation without using floating-point units. [4]
- **Sample-and-hold technique:**
  Capturing and holding sensor values for a fixed period ensures that the data remains stable while processing, preventing glitches caused by rapidly changing readings.

These strategies collectively enable the system to accurately compute and display orientation angles in real time while minimizing hardware resource usage.

3.2 Design Method (PO(a)):

3.2.1 Workflow:

The overall system is implemented as a multi-stage signal processing pipeline, where each module performs a specific task. The workflow ensures reliable acquisition, filtering, and interpretation of accelerometer data in real time. There are 5 sections for our project. These are followings:

- Spi Master (i2c interfacing)
- Sample Hold
- Moving Average
- Roll
- Pitch

3.2.2 Spi Master (I2C Interfacing):

The SPI Master module is responsible for establishing communication between the Artix A7 chip of the Nexys A7-100T FPGA and the onboard 3-axis accelerometer ADXL326 [5]. It performs both the initialization of the sensor and continuous acquisition of acceleration data along the X, Y, and Z axes. This module serves as the primary data acquisition unit of the system.

The Serial Peripheral Interface (SPI) protocol is used for communication. It consists of four signals: Master Out Slave In (MOSI), Master In Slave Out (MISO), Serial Clock (SCLK), and Chip Select (CS). The FPGA operates as the master device, while the accelerometer acts as the slave.

The SPI master operates using a derived clock signal. The system clock of 100 MHz is first reduced to 4 MHz using a clock divider. From this 4 MHz clock, a 1 MHz SPI clock is generated by toggling a register at regular intervals [6]. This ensures proper timing for reliable communication with the accelerometer.

The functionality of the SPI master is controlled by a finite state machine (FSM). The operation can be divided into four main phases: power-up delay, sensor configuration, data stabilization wait, and continuous data read.

Initially, in the power-up phase, the system waits for approximately 6 milliseconds to allow the accelerometer to stabilize after power is applied. This delay ensures that the sensor is ready to receive configuration commands.

In the configuration phase, the SPI master sends a sequence of commands to the accelerometer to enable measurement mode. First, a write instruction is transmitted, followed by the address of the power control register. Finally, a control value is sent to activate measurement mode. All data is transmitted serially through the MOSI line, one bit at a time, synchronized with the SPI clock.

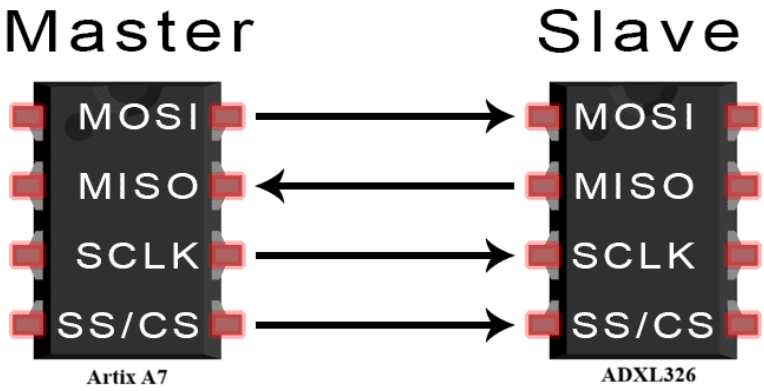

Fig 3.3: I2C Interfacing

After configuration, the system enters a waiting phase of approximately 40 milliseconds. This delay allows the accelerometer to begin generating valid measurement data.

Following initialization, the SPI master continuously performs read operations. A read instruction is sent along with the starting register address corresponding to the least significant byte (LSB) of the X-axis data. The accelerometer automatically increments the register address, allowing sequential reading of all six bytes corresponding to X, Y, and Z axes.

The received data is captured serially through the MISO line. Each byte is reconstructed bit by bit and stored in temporary registers. Two consecutive bytes are combined to form a 16-bit signed value for each axis. This process is repeated for X, Y, and Z data.

To ensure stable output, the acquired data is latched into dedicated registers after the completion of a full read cycle. This prevents inconsistencies that may arise from partial updates during ongoing SPI communication. A data_valid signal is generated to indicate that new, complete data is available for further processing.

The chip select (CS) signal is actively controlled to define the start and end of each SPI transaction. It is set low during communication and high when the transmission is complete. The serial clock (SCLK) is enabled only during active communication to maintain proper SPI timing.

After completing one full data acquisition cycle, the state machine returns to the read state, enabling continuous sampling of accelerometer data at a rate of approximately 100 Hz. This continuous loop ensures real-time data availability for subsequent processing stages such as filtering and angle computation.

In summary, the SPI master module performs sensor initialization, serial-to-parallel data conversion, and synchronized data delivery. It forms the foundation of the entire system by providing accurate and timely acceleration data for further processing and display.

3.2.3 Signal Stabilization:

The raw data obtained from the accelerometer is inherently noisy and continuously varying due to environmental disturbances, sensor imperfections, and quantization effects. Direct use of this data in angle computation may lead to unstable and inaccurate results. Therefore, a signal stabilization stage is introduced in the system, consisting of two key modules: Sample-and-Hold and Moving Average Filter. These modules ensure that the input to the angle computation block is stable, synchronized, and noise reduced.

A. Sample-and-Hold Operation:

The Sample-and-Hold module is responsible for capturing and maintaining a stable snapshot of the accelerometer data for a fixed duration. Since the SPI master continuously updates the X, Y, and Z values, there exists a possibility of intermediate transitions during processing. The Sample-and-Hold block eliminates this issue by synchronizing the data flow.

The module operates based on a control signal named **data valid,** which is generated by the SPI master after completing a full data acquisition cycle. When this signal is asserted, the current values of the accelerometer outputs (X, Y, Z) are captured and stored in internal registers. These stored values are then held constant until the next valid data sample is received.

This mechanism ensures that all downstream processing blocks operate on a consistent set of data corresponding to the same time instant. Without this stage, different modules might process partially updated values, leading to incorrect angle calculations.

Additionally, the Sample-and-Hold module generates a one-clock-cycle pulse (`sh_valid`) to indicate that a new stable sample has been captured. This pulse is used to trigger subsequent processing stages, ensuring proper synchronization throughout the pipeline.

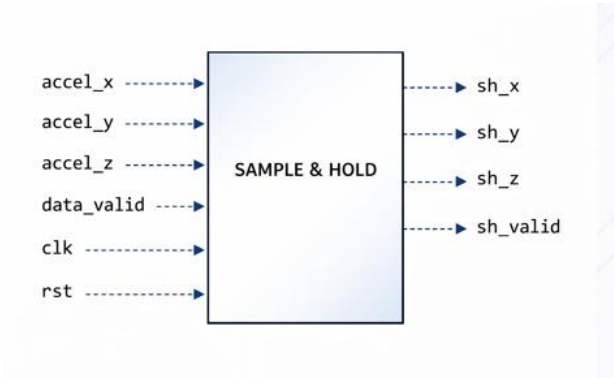

Fig 3.4: Sample and Hold Block Diagram

Key Functions of Sample-and-Hold:

- Synchronizes data acquisition with processing stages
- Eliminates glitches caused by continuous updates
- Provides time-aligned X, Y, Z data
- Generates a control signal for downstream modules

B. Moving Average Filter:

Although the Sample-and-Hold module stabilizes the timing of the data, it does not remove noise present in the accelerometer readings. To address this, a Moving Average Filter is implemented to smooth the data.

The moving average filter computes the average of a fixed number of recent samples. In this design, an 8-point moving average filter is used. The output at any given instant is the average of the current sample and the previous seven samples. This significantly reduces high-frequency noise while preserving the overall trend of the signal.

Mathematically, the moving average filter is expressed as:

$$Avg_{out} = \frac{x[n]+x[n-1]+...+x[n-(N-1)]}{N}$$

New Sum = Old Sum + New Sample − Oldest Sample

where:

- x[n] is the input signal
- Avgout is the filtered output
- N is the number of samples (in this case, 8)

In hardware implementation, instead of recalculating the sum for every new sample, an efficient **running sum technique** is used. The filter maintains a shift register containing the last N samples and a running sum of these values. When a new sample arrives:

- The oldest sample is removed from the sum
- The new sample is added to the sum
- The average is computed by dividing the updated sum by N

Since N is chosen as a power of 2 (N = 8), the division operation is efficiently implemented as a right bit shift, reducing hardware complexity.

The filter updates only when the **sh_valid** signal is asserted, ensuring that only valid and synchronized samples are processed.

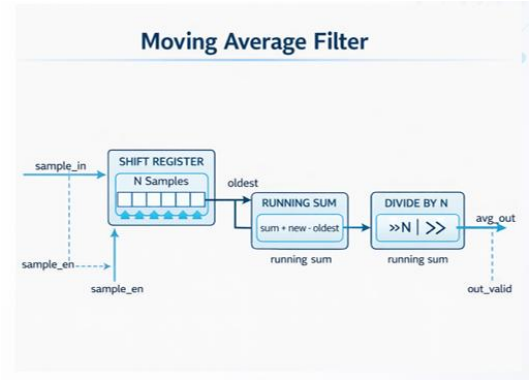

Fig 3.5: Moving Average Filter Block Diagram

Key Functions of Moving Average Filter:

- Reduces random noise in accelerometer data
- Produces smooth and stable signals
- Improves accuracy of angle computation
- Efficient hardware implementation using shift registers and adders

C. Combined Effect of Signal Stabilization

The combination of Sample-and-Hold and Moving Average filtering provides a robust signal conditioning stage. The Sample-and-Hold ensures temporal stability, while the Moving Average filter ensures amplitude stability.

As a result:

- Sudden spikes and fluctuations are minimized
- Data becomes smooth and reliable
- Angle computation using CORDIC becomes more accurate
- Display outputs (LEDs and 7-segment) become stable and visually consistent

This signal stabilization stage plays a crucial role in enhancing the overall performance and reliability of the system.

3.2.4 CORDIC Algorithm for Angle Computation

The CORDIC (Coordinate Rotation Digital Computer) algorithm is used in this project to compute the roll and pitch angles from the filtered accelerometer data. It is highly suitable for FPGA implementation because it avoids complex operations such as multiplication and division, and instead uses only shift and addition operations.

The algorithm works by iteratively rotating a vector in a coordinate system. In vectoring mode, it determines the angle of a vector given its Cartesian components. In this system, the accelerometer outputs are treated as vector components. The roll angle is computed using the Y and X components, while the pitch angle is computed using the X and Z components.

The relationships are given by:

$\theta_roll = \tan^{-1}(Y / X)$
$\theta_pitch = \tan^{-1}(X / Z)$

Fixed-Point Representation (Q1.15 Format)

Since floating-point operations are inefficient in FPGA hardware, fixed-point representation is used. In this design, the Q1.15 format is adopted, where one bit represents the sign and the remaining fifteen bits represent the fractional part. [7]

Using this format, angle values are scaled and represented as 16-bit signed integers. The range is mapped such that:

- $+\pi$ corresponds approximately to +32767
- $-\pi$ corresponds approximately to −32768

This allows efficient storage and processing of angle values in hardware without requiring floating-point units.

Lookup Table (LUT) Interpretation

To interpret the CORDIC output, lookup tables are used to relate the fixed-point values to actual angle ranges. Two separate LUTs are considered for pitch and roll angles.

For the pitch angle, the range is limited to −90° to +90°. The corresponding fixed-point values are scaled proportionally. For example, −90° corresponds to −16384, 0° corresponds to 0, and +90° corresponds to +16384. Intermediate values such as ±22.5°, ±45°, and ±67.5° are also included to provide better resolution. These values are used as thresholds for activating different LED levels in the pitch detection module.

LUT for Pitch angle

Angle (deg)	Signed 16-bit (Q15)
−90°	−16384
−67.5°	−12288
−45°	−8192
−22.5°	−4096
0°	0
22.5°	4096
45°	8192
67.5°	12288
90°	16384

Fig 3.6: Look up Table for Pitch Angle

For the roll angle, the range extends from −180° to +180°. Therefore, the full 16-bit signed range is utilized. In this mapping, −180° corresponds to −32768, 0° corresponds to 0, and +180° corresponds to +32767. Intermediate angles such as ±45°, ±90°, and ±135° are also represented. These values are used to define threshold regions for controlling the rotation pattern displayed on the 7-segment display.

LUT for Roll angle

Angle (deg)	Angle (rad)	Signed 16-bit (Q15)
−180°	$-\pi$	−32768
−135°	$-3\pi/4$	−24576
−90°	$-\pi/2$	−16384
−45°	$-\pi/4$	−8192
0°	0	0
45°	$\pi/4$	8192
90°	$\pi/2$	16384
135°	$3\pi/4$	24576
180°	π	32767

Fig 3.7: Look up Table for Roll Angle

Implementation

The output of the CORDIC module is a 16-bit signed value in Q1.15 format. Instead of converting this value into degrees, the system directly compares it with predefined threshold values obtained from the lookup tables.

For roll angle computation, the thresholds determine which pattern is displayed on the 7-segment display, creating a visual rotation effect. For pitch angle computation, the thresholds determine how many LEDs are activated and whether the tilt is forward or backward.

3.2.5 Tilt-Roll Angle Representation

After computing the roll and pitch angles using the CORDIC algorithm, the resulting fixed-point values are mapped to visual outputs. In this system, the roll angle controls the rotation of a character on the 7-segment display, while the pitch angle controls an LED-based level meter.

A. Roll Angle to 7-Segment Display Mapping

The roll angle represents rotation around the X-axis and ranges from −180° to +180°. This range is divided into four regions to represent different orientations of the display character.

For angles between −45° and +45°, the system is considered to be in the neutral position. In this case, the character is displayed in its normal orientation (open side facing downward).

For angles between −45° and −135°, the system is tilted to the left. The displayed character is rotated to reflect this leftward tilt.

For angles between −135° and +135°, the system is in an inverted position. The character appears flipped on the display.

For angles between +45° and +135°, the system is tilted to the right. The character is rotated accordingly to indicate rightward tilt.

These angle regions are implemented using threshold comparisons of the CORDIC output in fixed-point (Q1.15) format. Based on the region, different 7-segment patterns are selected to simulate rotation.

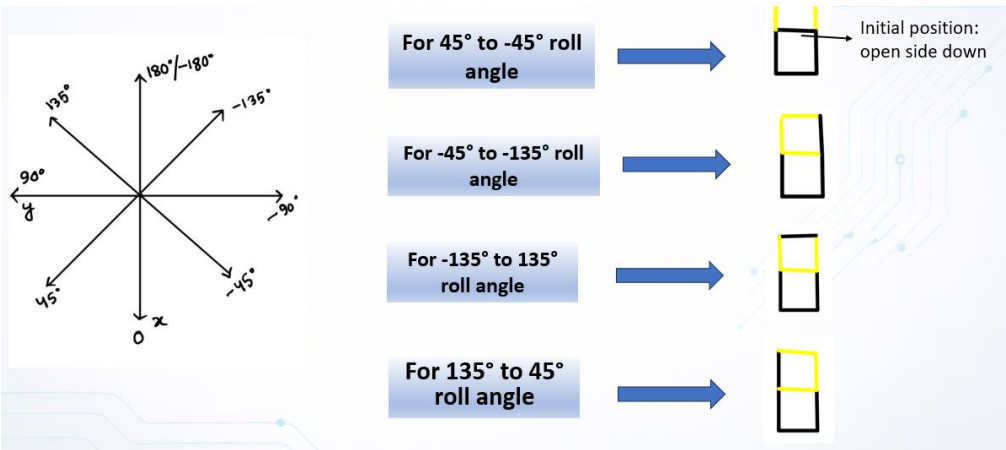

Fig 3.8: Roll Angle to Character Rotation

B. Pitch Angle to LED Level Meter Mapping

The pitch angle represents forward and backward tilt and is mapped to eight LEDs, forming a digital level meter. The LEDs indicate both the direction and magnitude of tilt.

For positive pitch angles (forward tilt), the following mapping is used:

- 0° to 22.5° activates LED[3]
- 22.5° to 45° activates LED[2]
- 45° to 67.5° activates LED[1]
- 67.5° to 90° activates LED[0]

As the tilt increases, LEDs turn on sequentially, indicating increasing magnitude.

For negative pitch angles (backward tilt), a separate set of LEDs is used:

- 0° to −22.5° activates LED[7]
- −22.5° to −45° activates LED[6]
- −45° to −67.5° activates LED[5]
- −67.5° to −90° activates LED[4]

Pitch Angle: LED level Meter

Pitch Angle	LED output
0°-22.5°	LED[3]
22.5°-45°	LED[2]
45°-67.5°	LED[1]
67.5°-90°	LED[0]
0°- -22.5°	LED[7]
-22.5°- -45°	LED[6]
-45°- -67.5°	LED[5]
-67.5°- -90°	LED[4]

Fig 3.9: Pitch Angle to LED Level Meter

This separation allows clear distinction between forward and backward tilt directions.

3.3 CAD Design:

The system architecture is as follows

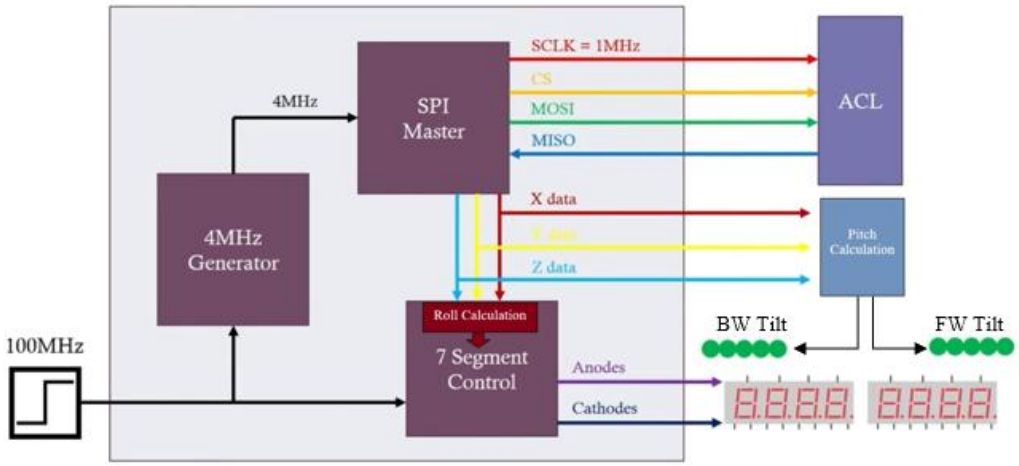

Fig 3.10: System Architecture

The CAD design for our project is the following

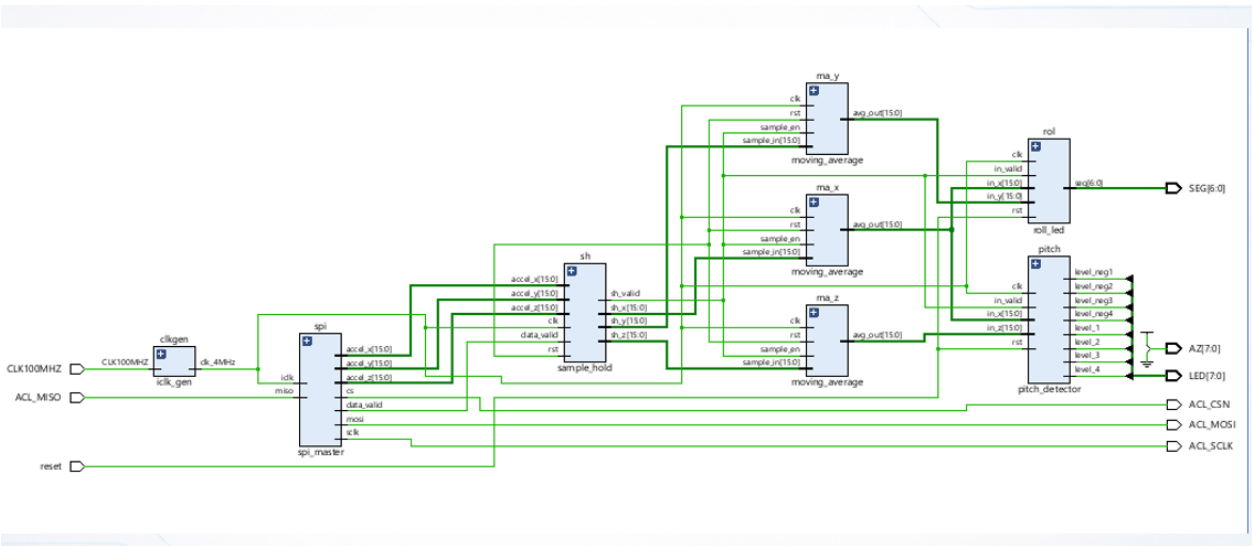

Fig 3.11: CAD Design

3.4 Full Source Code:

Project Summary | EEE304_Project.v *

C:/Users/ACER/Desktop/EEE 304 DLD Project/EEE304_Project/EEE304_Project.srcs/sources_1/new/EEE304_Project.v

```
`timescale 1ns/1ps

module EEE304_Project(
input [2:0] reset,
input CLK100MHZ,
input ACL_MISO,
output ACL_MOSI,
output ACL_SCLK,
output ACL_CSN,
output [7:0] LED,
output [7:0] AZ,
output [6:0] SEG
);

//-----------------------------------------------
// Clock
//-----------------------------------------------

assign AZ=8'b11111110;
wire clk_4MHz;
iclk_gen clkgen (
.CLK100MHZ (CLK100MHZ),
.clk_4MHz (clk_4MHz)
);
```

Fig 3.12: Code Snippet

(The remaining is added in the GitHub link)

4 Implementation

4.1 Description

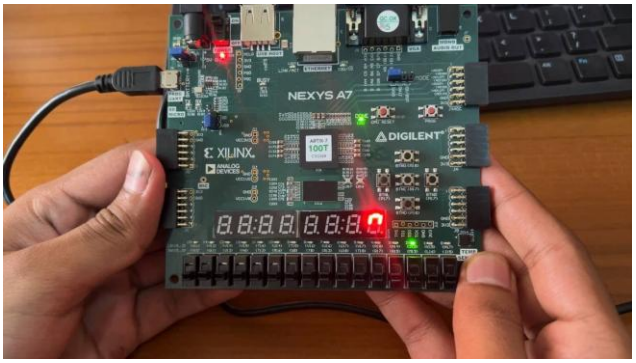

Fig 4.1: 0° Rotation

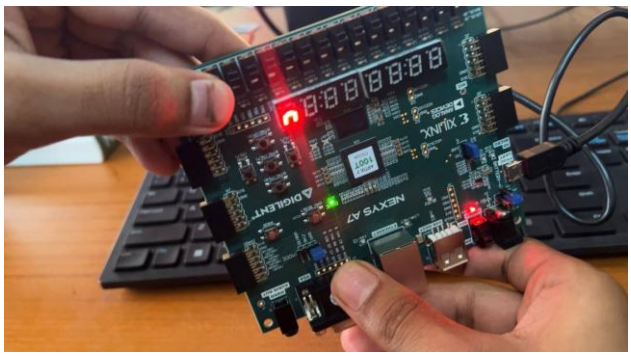

Fig 4.2: 180° Rotation

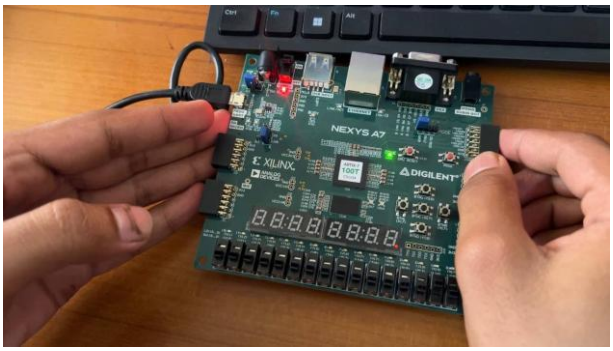

Fig 4.3: 0° Tilt

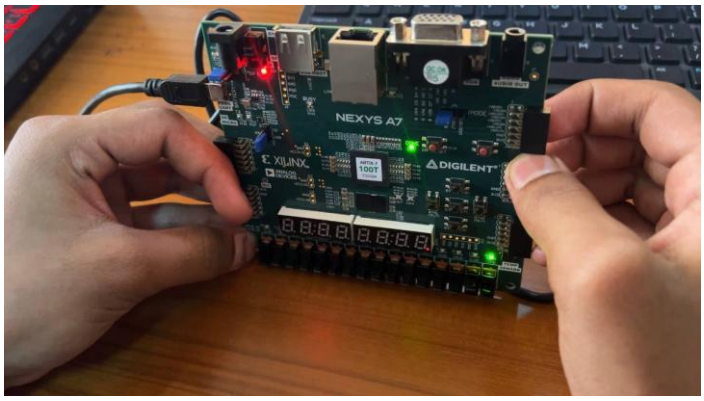

Fig 4.4: 90° Forward Tilt

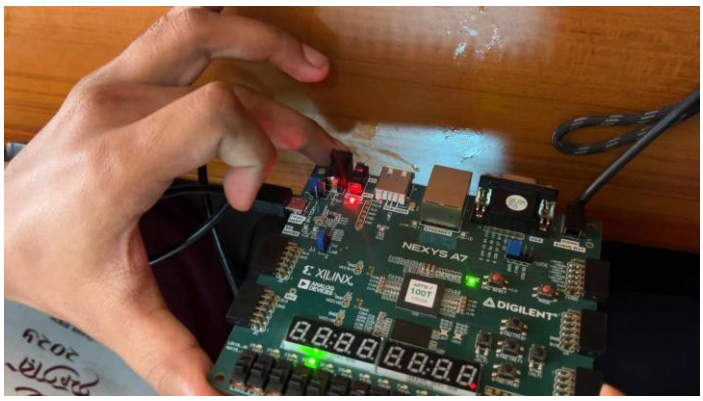

Fig 4.5: 90° Backward Tilt

5 Design Analysis and Evaluation

5.1 Novelty

This project presents a complete FPGA-based real-time orientation detection system that integrates multiple functional stages into a single hardware platform. The key novelty of the design lies in the seamless combination of sensor interfacing, signal processing, and real-time visualization within a unified architecture.
A significant contribution of this work is the implementation of the CORDIC (Coordinate Rotation Digital Computer) algorithm, which enables efficient angle computation without the need for complex multipliers or floating-point units. This makes the design highly suitable for FPGA-based realization.
Furthermore, the system provides immediate visual feedback by mapping physical tilt directly to LED indicators and 7-segment display outputs. This direct hardware-level interaction ensures fast response and demonstrates an effective real-time embedded system.

5.2 Design Considerations (PO(c))

Accuracy vs. Hardware Efficiency
To optimize hardware resource utilization, a 16-bit fixed-point representation is used instead of floating-point arithmetic. This significantly reduces computational complexity and power consumption, although it introduces minor approximation errors in angle calculation.
Noise Handling
Accelerometer outputs are inherently noisy due to environmental and electronic factors. To address this, a moving average filter is implemented, which smooths the input signals and enhances the stability of the system output.
Real-Time Operation
The system operates at an approximate sampling rate of 100 Hz, ensuring smooth and continuous tracking of orientation. A pipeline-based architecture is employed to allow uninterrupted data flow between modules, enabling efficient real-time processing.
Safety and Environmental Considerations
The design is implemented on a low-power FPGA platform, making it energy-efficient and safe for operation. It does not generate harmful emissions and is suitable for long-term use.

5.3 Investigations (PO(d))

5.3.1 Design of Experiment

The FPGA board was manually tilted at different angles along both pitch (forward/backward) and roll (left/right) directions to evaluate system performance under varying conditions.

5.3.2 Data Collection

- LED outputs were observed to monitor pitch angle variations.
- 7-segment display patterns were analyzed to evaluate roll angle changes.

5.3.3 Results and Analysis

- Without filtering, the output signals were unstable and exhibited noticeable noise.
- After applying the moving average filter, the system produced smoother and more stable outputs.
- The CORDIC algorithm provided consistent and reliable angle estimation across different tilt conditions.

5.3.4 Conclusion

The experimental results confirm that the system can accurately detect orientation changes and provide stable real-time feedback, validating the effectiveness of the overall design.

5.4 Limitations of Tools (PO(e))

Hardware Limitations

- Sensor readings are inherently noisy and require filtering for stable output.
- The accelerometer measurement range (typically ±2g to ±8g) limits the detection of extreme motion.
- The FPGA board has limited onboard peripherals, restricting system expansion without additional external modules.

Processing and Algorithmic Limitations

- Fixed-point arithmetic provides lower precision compared to floating-point calculations.
- The CORDIC algorithm introduces small approximation errors in angle computation.
- Incorporating more complex algorithms may affect real-time performance due to increased resource usage.
- The system evaluates roll and pitch independently and cannot fully represent complex multi-axis motion simultaneously.

5.5 Impact Assessment (PO(f))

5.5.1 Societal and Cultural Impact

- The system can be applied in robotics, smartphones, gaming systems, and interactive devices.

- It enhances understanding of motion sensing and embedded system design, contributing to educational and technological development.

5.5.2 Health and Safety Impact

- Tilt sensing systems can be used for monitoring structural stability in buildings, bridges, and towers by detecting abnormal movements.
- In automotive systems, tilt and acceleration sensors can help detect rollover conditions and activate safety mechanisms.
- In industrial environments, such systems can monitor heavy machinery to prevent unsafe operation and reduce accidents.

5.5.3 Legal Considerations

The project utilizes standard hardware components and does not involve any legal or regulatory restrictions.

5.6 Sustainability Evaluation (PO(g))

The system is designed with sustainability in mind:

- Low power consumption due to efficient FPGA implementation
- Reusable hardware platform for future applications
- Minimal component usage, reducing electronic waste

5.7 Ethical Issues (PO(h))

All design work and code implementation were developed by the project team. Any external references or resources used have been properly acknowledged, ensuring compliance with ethical standards and avoiding misuse of intellectual property.

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member

Student ID	Name	Contribution
2106181	**Mushfiqur Rahman**	SPI protocol, mathematical analysis, presentation slide, debugging, Report writing, Poster
2106182	**Chayan Paul**	System stabilization and filtering, Mathematical analysis, Debugging, Presentation slide, Report writing
2106183	**Shamin Yeaser**	Mathematical analysis, Angle calculation, Tilt meter logic implementation, Combining individual codes, Debugging
2106184	**Kayes Sami Malitha**	Display rotator logic implementation, Mathematical analysis, Debugging, Presentation slide, Report writing

6.2 Mode of Team Work

- Tasks were divided based on individual strengths in FPGA design and digital logic
- Regular meetings were held to review progress
- Code integration and debugging were performed collaboratively
- Team members supported each other during testing and troubleshooting

6.3 Diversity Statement of Team

The team consisted of members with different technical expertise and skills in digital system design, FPGA programming, and data analysis. This diversity helped in approaching the problem from multiple perspectives and contributed to the successful implementation of the display rotator and tilt level meter system. Collaboration among team members allowed effective problem-solving and improved the overall quality of the project.

6.4 Log Book of Project Implementation

Week	Milestone achieved	Individual Role	Team Role	Comments
Week 4	Literature review on accelerometer-based roll pitch calculation and SPI protocol completed	2106183 reviewed angle calculation, and 2106181 reviewed SPI protocol implementation	Team discussed system architecture	Design approach finalized
Week 6	SPI protocol implemented on FPGA	2106181 implemented the SPI master module	Team verified accelerometer data	Accelerometer data acquisition completed
Week 7	Roll and pitch angle calculation implemented	2106183 implemented the CORDIC algorithm and the overall angle calculation code	Team verified the orientation logic and determined the X, Y & Z axis of the FPGA	Roll pitch angle calculated and verified
Week 8	Filtering method for stabilization of the raw accelerometer data implemented	2106182 implemented the sample hold and moving average module	Team verified correct operation	Stable data output confirmed
Week 9	Display rotation implemented on the 7-segment display	2106184 implemented the code	Team verified correct operation and helped fix logical corrections	Display rotator implemented
Week 10	Tilt level meter implemented using LEDs	2106183 implemented the logic, and 2106182 helped with the code	Team verified correct operation	Tilt level meter implemented
Week 11	All module merging and testing	2106181 and 2106184 merged the code	Team observed hardware testing	Full system implemented
Week 12	Final debugging and report preparation	All members participated	Team finalized documentation and results	Project completed

7 Communication to External Stakeholders (PO(j)).

7.1 Executive Summary

This project presents the design and implementation of a real-time tilt detection and display rotation system using the Nexys A7 FPGA platform and its onboard accelerometer. The system acquires acceleration data through SPI communication, processes it using digital filtering techniques, and computes orientation angles using a hardware-efficient CORDIC algorithm. The resulting tilt and roll information is displayed using LEDs and a 7-segment display, providing intuitive visual

feedback. The design demonstrates efficient hardware-based signal processing without the use of a microcontroller. Such a system has potential applications in leveling instruments, motion sensing devices, and embedded orientation monitoring systems.

7.2 User Manual

- Connect the Nexys A7 FPGA board to a computer using a USB cable and ensure that the board is properly powered.
- Open the FPGA design tool (e.g., Vivado) and program the board with the generated bitstream file corresponding to the project.
- Once the programming process is complete, the system will automatically begin operation without requiring any external input or configuration.
- The onboard accelerometer will continuously measure acceleration along the X, Y, and Z axes, and the data will be processed internally in real time.
- Place the board on a flat surface to observe the reference (neutral) condition of the system.
- Gently tilt the board along the horizontal axis (left or right direction):
    - The 7-segment display will change its output pattern.
    - Different segment configurations correspond to different roll angle ranges.
- Tilt the board along the vertical axis (forward or backward direction):
    - The onboard LEDs will indicate the pitch angle.
    - The number and position of illuminated LEDs represent the magnitude and direction of tilt.
- For small angular deviations, fewer LEDs will be activated, whereas larger tilt angles will result in more LEDs being illuminated based on predefined thresholds.
- When the board is returned to its neutral position, both the LED indicators and 7-segment display will revert to their default states.
- The system operates continuously and updates dynamically, ensuring immediate response to any change in orientation.

7.3 Github Link

GitHub - intmushfiq/FPGA-Tilt-Meter-Display-Rotator: This project uses an onboard accelerometer, ADXL362, on the Nexys A7 100T FPGA board to determine device tilt and orientation and provide real-time visual feedback. · GitHub

7.4 YouTube Link

https://www.youtube.com/watch?si=wJ8-o-TuKVfmG2kN&v=dHNygxtZgfk&feature=youtu.be

8 Future Work (PO(l))

- The system can be enhanced by adding wireless communication so that tilt data can be monitored remotely.
- Accuracy can be further improved by using more advanced filtering techniques instead of the current method.
- A real-time alert feature can be introduced, such as a buzzer or mobile notification, to warn users during unsafe conditions.
- The design can be made more compact and portable for practical industrial use.
- In the future, this system may be applied in real-life areas like earthquake monitoring, industrial safety systems, and vehicle stability control.

9 References

[1] Collins, D. (2020, May 26). Motion basics: How to define roll, pitch, and yaw for linear systems. Retrieved from https://www.linearmotiontips.com/motion-basics-how-to-define-roll-pitch-and-yaw-for-linear-systems/

[2] Al-Sahly, Abdullah & Hassan, Mohammad & Saleem, Kashif & Alabrah, Amerah & Rodrigues, Joel. (2022). Handheld Device-Based Indoor Localization with Zero Infrastructure (HDIZI). Sensors. 22. 10.3390/s22176513.

[3] Implementing the Moving Average (Boxcar) filter. (2017, October 16). Retrieved from https://zipcpu.com/dsp/2017/10/16/boxcar.html

[4] P.Trujillo. (2022, April 24). Exploring the Cordic algorithm. Retrieved from https://www.controlpaths.com/2022/04/25/exploring-the-cordic-algorithm/

[5] Brown, A. (2025, June 12). NeXyS A7 Reference Manual - Digilent Reference. Retrieved from https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual?srsltid=AfmBOor00iHNWqy4nsOLaHzHqumgkCucfRaJSsK7_0mbGvvdHReyxpJm

[6] Karan-nevage/Nexys-A7-Accelerometer-FPGA: This project showcases the integration of a 3-axis accelerometer (ADXL362) with the Nexys A7-50T FPGA board. The accelerometer data is read via SPI communication and displayed on 7-segment displays and LEDs. The Verilog code handles SPI communication, data processing, and display control. (n.d.). Retrieved from https://github.com/Karan-nevage/Nexys-A7-Accelerometer-FPGA

[7] Wikipedia contributors. (2025, August 16). Q (number format). Retrieved from https://en.wikipedia.org/wiki/Q_(number_format)